



**UNIVERSITY
OF LONDON**

University of London BSc Computer Science

CM3045: 3D graphics & Animation

Student Name: Tan Liang Ting

Student Registration Number (SRN): 230656747

Overview

This project is a tennis themed Unity 6 scene built around a physics-controlled ball. The scene meets the baseline by using Unity rigid bodies and colliders. I implemented a physics object that behaves under simulation, force based WASD movement for the player, and a third person camera that orbits using the mouse. I also added collectable tennis balls that open the route by removing a gate once all tennis balls are collected.

Third person orbit camera (mouse)

I implemented a playable third person camera that stays behind the action, rotates with the mouse, and does not clip through obstacles when the ball moves near walls or the gate.

Yaw and pitch orbit with clamping. The camera stores yaw and pitch values updated by mouse input. Pitch is clamped ($\text{minPitch} = -20$, $\text{maxPitch} = 70$) so the camera cannot flip upside down and does not aim straight into the floor.

Quaternion rotation. The orbit rotation is built using `Quaternion.Euler(pitch, yaw, 0)` and applied to a backwards offset to place the camera on a sphere around the target. This avoids gimbal problems that appear when stacking Euler rotations manually.

Smoothing. I use a smoothing time of about 0.08 so the camera follows without snapping. This reduces jitter that is noticeable when the ball is rolling and colliding.

Collision avoidance (SphereCast). I use a sphere cast from the target pivot to the desired camera position ($\text{collisionRadius} = 0.2$). If it hits geometry, I pull the camera closer using a small buffer (0.1). This prevents the camera from going through the gate or obstacles.

The orbit camera makes the scene feel like a sports third person game rather than a fixed demo view. I implemented it directly instead of using Cinemachine because the maths and logic are easy to explain and mark. The sphere cast is important in my scene because obstacles and the gate can sit between the player and the camera. Without collision handling, the camera clips through objects and the player loses visual feedback.

Extension: Collectable tennis balls that opens a gate

The extension adds a simple tennis themed objective: the player must collect all tennis balls placed around the court. Once all balls are collected, the blocking gate will open so the player can roll into the next area. This creates a clear “drill completion” loop and gives the scene a visible success condition.

Trigger-based interaction

Each collectable uses a trigger collider, so collection is handled by the physics system when the player enters the trigger volume. This is implemented using `OnTriggerEnter`,

which fires on overlap between a trigger collider and the player's collider. This approach avoids manual distance checks and scales well because it only runs when an overlap occurs.

State management and threshold logic.

A manager object tracks progress using two integers: collected and requiredToUnlock. Each pickup increments the count by 1. When collected $\geq$ requiredToUnlock, the system triggers a gate change. In logic terms, the extension is a simple finite state change:

- Before completion: collected $<$ requiredToUnlock $\rightarrow$ gate remains active
- Completion: collected $\geq$ requiredToUnlock $\rightarrow$ gate removed

This makes the completion condition explicit and easy to verify during marking.

Gate removal by disabling renderers and colliders.

In the final version, the gate will be opened by disabling its visual and physical components:

- disabling the Renderer(s) makes it disappear visually
- disabling the Collider(s) removes collision so the player can pass through

This is a stable approach because it avoids hinge instability and “door nudging” from physics at the start of Play mode.

I used triggers for collectables because it is the most reliable Unity method for pickups and it keeps the interaction consistent with the physics-based player. The manager-based counter is easy to debug and demonstrate, since the number of collected tennis balls directly maps to the unlock event.

For the gate, I chose “disappear on completion” rather than a swinging hinge door. A hinge door can look good, but it is also easy to break with a slightly wrong pivot, overlapping colliders, or initial physics settling. Disabling colliders guarantees the path becomes open exactly when the tennis balls are collected. This prioritises predictable behaviour and clear evidence for assessment.

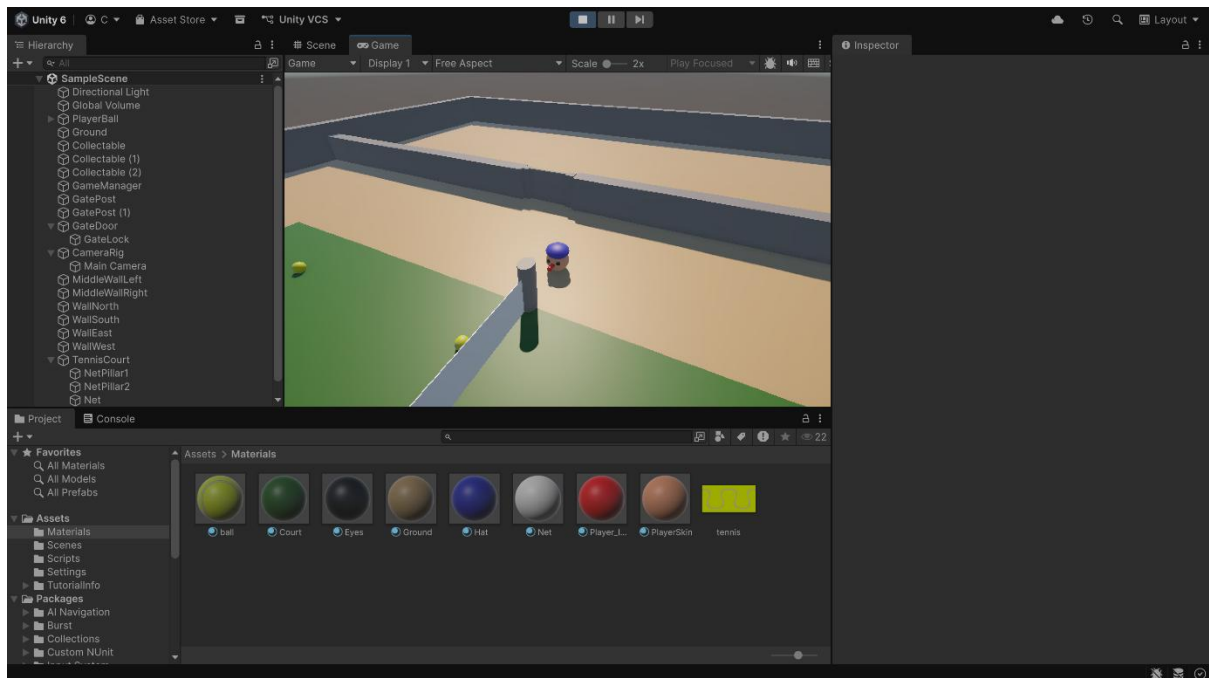


Figure 1: Gate close when tennis balls are not collected

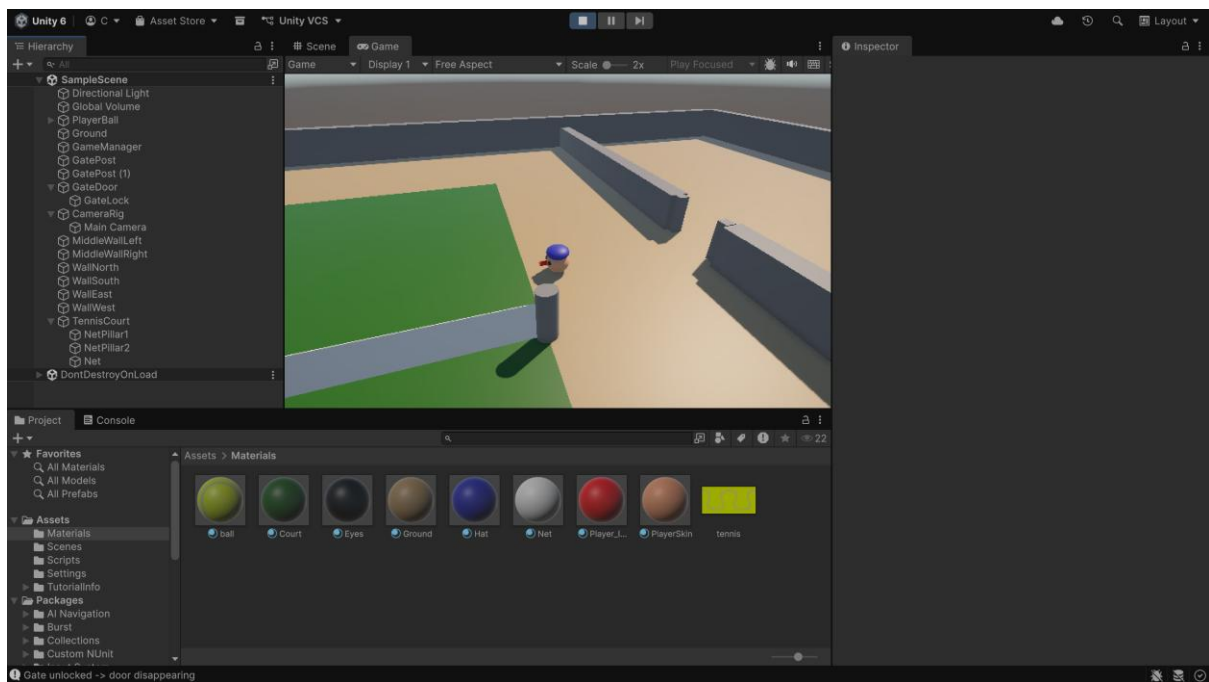


Figure 2: Gate open when all tennis balls are collected

References

Unity Technologies (n.d.) *Unity Scripting API: Collider.OnTriggerEnter, Rigidbody, Rigidbody.AddForce, Physics.Raycast, Physics.SphereCast, Quaternion, Vector3.SmoothDamp*. Unity Documentation

Unity Technologies (n.d.) *Unity Manual: Colliders, Triggers, Rigidbody component and Physics*. Unity Documentation.

Paul Bourke, “Of spherical nature,” *Paul Bourke – Personal Pages* (January 2017).